

Laboratorium: porównanie algorytmów regresji

Uczenie nadzorowane | Python | scikit-learn

1. Cel zajęć

Celem laboratorium jest rozwiązanie problemu regresji oraz porównanie kilku algorytmów uczenia nadzorowanego na tym samym zbiorze danych. Studenci uruchomią gotowy kod, przeanalizują wyniki, zmienią wybrane parametry modeli i przygotują krótkie wnioski.

2. Problem regresji

Zadanie polega na przewidywaniu zużycia paliwa samochodu w jednostce MPG, czyli miles per gallon. Im większa wartość MPG, tym bardziej ekonomiczny samochód. Jest to problem regresji, ponieważ zmienna docelowa jest liczbą rzeczywistą.

3. Dane

W ćwiczeniu używany jest publiczny zbiór Auto MPG z UCI Machine Learning Repository. Dane dotyczą samochodów z lat 1970-1982 i zawierają cechy techniczne, takie jak liczba cylindrów, pojemność silnika, moc, masa, przyspieszenie, rok modelowy oraz pochodzenie pojazdu.

- Zmienna docelowa: mpg.
- Cechy wejściowe: cylinders, displacement, horsepower, weight, acceleration, model_year, origin.
- Kolumna car_name jest w kodzie pomijana jako identyfikator/nazwa pojazdu.
- W kolumnie horsepower występują braki danych oznaczone znakiem ?. Kod uzupełnia je medianą.

4. Wymagane środowisko

Przed zajęciami należy przygotować środowisko Python 3.10 lub nowsze. Wymagane biblioteki można zainstalować poleceniem:

```
pip install pandas numpy matplotlib scikit-learn
```

5. Pliki przekazane studentom

- regresja_auto_mpg_lab.py - gotowy program Python do uruchomienia i modyfikacji.
- instrukcja_laboratorium_regresja_auto_mpg.docx - niniejsza instrukcja.
- prezentacja_algorytmy_regresji.pptx - prezentacja omawiająca zastosowane algorytmy.

6. Harmonogram zajęć - 135 minut

Czas	Etap	Opis aktywności
0-10 min	Wprowadzenie	Regresja, zmienna docelowa, metryki MAE, RMSE i R2.
10-25 min	Uruchomienie środowiska	Sprawdzenie bibliotek,

		uruchomienie skryptu, pobranie danych z internetu.
25-45 min	Analiza danych	Obejrzenie pierwszych wierszy, braków danych i cech wejściowych.
45-75 min	Trening modeli	Regresja liniowa, Ridge, Lasso, k-NN, drzewo, Random Forest, Gradient Boosting, SVR.
75-100 min	Ewaluacja	Porównanie modeli według MAE, RMSE, R2 i walidacji krzyżowej.
100-120 min	Modyfikacje studentów	Zmiana parametrów, dodanie modelu lub modyfikacja cech.
120-135 min	Wnioski	Krótką interpretacją wyników i dyskusją, który model jest najlepszy i dlaczego.

7. Zadania do wykonania

1. Uruchom program `regresja_auto_mpg_lab.py` i sprawdź, czy utworzył katalog `wyniki_regresji`.
2. Otwórz plik `wyniki_modeli_regresji.csv` i wskaż model z najniższym `RMSE_test`.
3. Porównaj trzy metryki: `MAE_test`, `RMSE_test` i `R2_test`. Napisz, co mierzy każda z nich.
4. Obejrzyj wykres `porownanie_modeli_rmse.png`. Czy ranking modeli jest zgodny z tabelą wyników?
5. Obejrzyj wykres `predykcje_vs_rzeczywiste.png`. Wyjaśnij, co oznacza linia przerywana.
6. Obejrzyj wykres `reszty_najlepszego_modelu.png`. Sprawdź, czy reszty są równomiernie rozproszone wokół zera.
7. Zmień jeden parametr wybranego modelu, np. `n_neighbors` w k-NN, `max_depth` w drzewie albo `n_estimators` w Random Forest. Uruchom program ponownie i porównaj wyniki.
8. Dodaj do programu jeden nowy model regresji albo usuń jedną cechę wejściową. Opisz, czy wynik się poprawił czy pogorszył.
9. Na podstawie pliku `waznosc_cech_random_forest.png` wskaż trzy najważniejsze cechy dla przewidywania MPG.
10. Przygotuj krótkie podsumowanie: który model rekomendujesz i dlaczego?

8. Algorytmy użyte w kodzie

Algorytm	Główna idea	Co obserwować w wynikach
Regresja liniowa	Dopasowuje prostą lub hiperpłaszczyznę minimalizując błąd kwadratowy.	Dobry punkt odniesienia; może być zbyt prosty dla zależności nieliniowych.
Ridge	Regresja liniowa z karą L2 ograniczającą duże współczynniki.	Stabilniejsza przy współliniowości cech.
Lasso	Regresja liniowa z karą L1, która może zerować niektóre współczynniki.	Może działać jak prosta selekcja cech.
k-NN	Przewiduje na podstawie najbliższych obserwacji treningowych.	Wrażliwy na skalowanie i dobór liczby sąsiadów.
Drzewo decyzyjne	Dzieli przestrzeń cech na regiony według reguł if-then.	Łatwe do interpretacji, ale podatne na przeuczenie.

Random Forest	Uśrednia wiele drzew trenowanych na losowych podpróbkach danych i cech.	Często stabilny i skuteczny; daje ważność cech.
Gradient Boosting	Buduje drzewa sekwencyjnie, każde kolejne poprawia błędy poprzednich.	Często bardzo dobre wyniki; wymaga strojenia parametrów.
SVR RBF	Regresja wektorów nośnych z nieliniowym jądrem RBF.	Wrażliwy na skalowanie i parametry C, gamma, epsilon.

9. Oczekiwane rezultaty

- Tabela z porównaniem modeli według MAE, RMSE i R2.
- Wykres porównania RMSE dla wszystkich modeli.
- Wykres predykcji względem wartości rzeczywistych dla najlepszego modelu.
- Wykres reszt dla najlepszego modelu.
- Wykres ważności cech dla Random Forest.
- Krótki komentarz interpretujący wyniki.

10. Pytania kontrolne

- Dlaczego nie oceniamy modeli wyłącznie na zbiorze treningowym?
- Jaka jest różnica między MAE a RMSE?
- Co oznacza wysoka wartość R2?
- Dlaczego skalowanie cech jest ważne dla k-NN i SVR?
- Dlaczego drzewo decyzyjne może się przeuczyć?
- Czym Random Forest różni się od pojedynczego drzewa decyzyjnego?
- Po czym poznać, że model systematycznie zawyża lub zaniża predykcje?

11. Kryteria zaliczenia

Element	Opis	Punkty
Uruchomienie kodu	Program działa i generuje pliki wynikowe.	2
Analiza tabeli wyników	Student poprawnie wskazuje najlepszy model i interpretuje metryki.	3
Analiza wykresów	Student potrafi wyjaśnić predykcje, reszty i ważność cech.	3
Modyfikacja eksperymentu	Student zmienia parametr, dodaje model lub modyfikuje cechy i porównuje wyniki.	4
Wnioski	Student formułuje krótką, logiczną rekomendację modelu.	3